Position Paper: AutoML Multiagéntico para la Generación de Pipelines Auditables y Explicables

Jonnathan Cuzco, Joshua Cueva, Juan Coronel, Víctor Saquicela
Universidad de Cuenca
{jonathan.cuzcot, joshua.cueva, juan.coronelc, victor.saquicela}@ucuenca.edu.ec

***Resumen*—Las herramientas actuales de automatización del aprendizaje automático (AutoML) aceleran la selección de algoritmos y la optimización de hiperparámetros, pero operan fundamentalmente como cajas negras: no razonan semánticamente sobre los datos, no justifican sus decisiones algorítmicas y carecen de la trazabilidad exigida por entornos altamente regulados. Este artículo de posición propone un sistema AutoML multiagéntico que integra agentes MCP especializados basados en LLMs, ontologías formales, y aprendizaje por refuerzo jerárquico para generar pipelines de ML con explicabilidad. La propuesta opera mediante dos fases complementarias: un entorno de simulación estocástica donde los agentes exploran iterativamente, y acumulan conocimiento empírico para entrenar políticas de aprendizaje por refuerzo; y una fase de producción determinista que ejecuta la estrategia aprendida con total transparencia. Crucialmente, la arquitectura incorpora agentes XAI y un agente generador de reportes que integra cada decisión y hallazgo en un reporte interpretable. Esta propuesta redefine el estado del arte, sentando las bases para una IA autónoma que no solo optimiza métricas de rendimiento, sino que justifica sus decisiones y audita sus propias acciones.**

***Index Terms*—AutoML, Sistemas Multiagente, Ontologías, Aprendizaje por Refuerzo, XAI, LLM, GraphRAG.**

1. Introducción

En el mundo digital contemporáneo, los datos constituyen el activo subyacente de mayor valor estratégico. El Machine Learning (ML) se ha posicionado como una tecnología fundamental para extraer conocimiento accionable de estos repositorios masivos de datos [11], impulsando decisiones en diagnósticos médicos, modelos de riesgo financiero, predicciones del clima y un sinfín de dominios donde la identificación de patrones latentes define el éxito competitivo [2].

Sin embargo, la construcción de modelos de ML de grado industrial sigue siendo un proceso altamente artesanal. Requiere seleccionar algoritmos idóneos entre cientos de candidatos, diseñar características complejas, ajustar hiperparámetros de forma minuciosa y validar cada decisión arquitectónica [6]. Este ciclo de desarrollo iterativo consume enormes recursos computacionales y temporales de ingenieros altamente especializados, está sujeto a sesgos heurísticos humanos y, lo que es más crítico para la industria, raramente genera una bitácora auditable que justifique por qué se eligió una metodología [27].

La automatización del ML (AutoML) surgió con la promesa de eliminar este cuello de botella [16]. Plataformas consolidadas como Auto-Sklearn [9], AutoGluon [7] y Auto-WEKA [19] han logrado sistematizar la búsqueda de pipelines mediante optimización bayesiana y meta-aprendizaje, liberando a los científicos de datos de la exploración manual. No obstante, tras más de una década de maduración tecnológica, el paradigma del AutoML clásico exhibe tres limitaciones estructurales severas que impiden su adopción masiva en ecosistemas empresariales de misión crítica.

Primero, estas herramientas tratan los conjuntos de datos como matrices numéricas, ignorando la semántica del dominio. Aunque un problema de abandono de clientes y uno de falla estructural de turbinas puedan parecer similares matemáticamente, requieren tratamientos distintos desde el punto de vista semántico [3]. Segundo, el AutoML tradicional funciona como una caja negra; el usuario recibe un modelo optimizado y una métrica de precisión, pero no conoce las transformaciones aplicadas ni las relaciones que el sistema utilizó [10]. Tercero, muchas herramientas generan arquitecturas rígidas y poco transparentes [30], incompatibles con los requisitos de Inteligencia Artificial Explicable (XAI) y con las exigencias de gobernanza algorítmica en sectores como el corporativo, financiero y de la salud.

De manera simultánea, la inteligencia artificial está experimentando una transición paradigmática impulsada por los Modelos de Lenguaje de Gran Escala (LLMs). Estos modelos han evolucionado de simples motores probabilísticos de texto a agentes autónomos dotados de capacidades de planificación, uso de herramientas y razonamiento complejo [25], [28]. Ecosistemas vanguardistas como AutoGen [26] demuestran que la orquestación de múltiples agentes especializados, interactuando bajo protocolos semánticos estrictos, permite resolver problemas intrincados minimizando las alucinaciones inherentes a los modelos monolíticos [15].

Este artículo de posición propone una arquitectura multiagente para evolucionar la automatización del aprendizaje automático. En lugar de limitarse a una optimización ciega, el sistema propuesto razona lógicamente sobre los datos mediante un motor ontológico, experimenta autónomamente en un entorno seguro y aprende de sus interacciones mediante aprendizaje por refuerzo jerárquico (HRL). En su fase de producción, genera pipelines deterministas con trazabilidad total y redacta informes narrativos que explican las decisiones tomadas y las variables clave del modelo. El resultado es un sistema preparado para el escrutinio empresarial, capaz de transicionar el AutoML desde la simple eficiencia algorítmica hacia la confianza, la explicabilidad y la autonomía auditable, facilitando la interpretabilidad de modelos de aprendizaje automático.

2. Escenario

En una empresa de distribución o comercio electrónico, decisiones como cuánto inventario mantener, qué precio asignar a un producto o cuándo ajustar el margen de una categoría se toman constantemente y pueden afectar directamente las ventas y la rentabilidad. Para apoyar estas decisiones, el AutoML permite automatizar la construcción de modelos predictivos y encontrar configuraciones que ofrecen un buen desempeño. Sin embargo, para un usuario del negocio, obtener una buena predicción no siempre es suficiente: también necesita entender por qué el sistema recomienda una determinada acción, qué factores influyeron en el resultado y si la recomendación es coherente con la realidad del negocio. Cuando estas soluciones no incorporan el conocimiento que los usuarios tienen sobre su dominio ni ofrecen explicaciones claras, resulta más difícil confiar en sus resultados y utilizarlos para tomar decisiones. Por ello, el desafío consiste en avanzar hacia sistemas de AutoML que no solo busquen modelos precisos, sino que también incorporen conocimiento del dominio y proporcionen explicaciones comprensibles y trazables. Aunque este problema se plantea en el contexto comercial, el mismo enfoque puede aplicarse a otros dominios donde las decisiones automatizadas requieren conocimiento especializado, confianza y capacidad de explicar cómo se llegó a una recomendación.

Para fundamentar científicamente la solución a este desafío, es necesario analizar la trayectoria tecnológica del AutoML y precisar la brecha estructural que impide a los sistemas actuales alcanzar esta visión.

Las primeras generaciones de AutoML buscaron resolver la Selección Combinada de Algoritmos y Optimización de Hiperparámetros (CASH) mediante optimización bayesiana y búsqueda heurística [23]. Herramientas como Auto-WEKA [19] demostraron que la búsqueda automatizada podía superar estrategias manuales en escenarios controlados. A partir de ahí, Auto-Sklearn incorporó meta-aprendizaje para aprovechar el conocimiento de conjuntos de datos similares [9], mientras que AutoGluon amplió el enfoque con ensamblajes multicapa orientados a maximizar la exactitud predictiva [7] y Auto-Keras lo llevó al ámbito de la búsqueda automática de arquitecturas neuronales [17]. En conjunto, estos avances demostraron que la automatización podía igualar e incluso superar el diseño humano de pipelines en determinados escenarios.

Sin embargo, la literatura reciente evidencia limitaciones estructurales en estos enfoques [3], [2]. Aunque son optimizadores altamente eficientes, operan principalmente como procesos de búsqueda secuenciales con una comprensión limitada del contexto del problema. No incorporan razonamiento semántico sobre el dominio, presentan dificultades para adaptarse a restricciones externas y carecen de mecanismos robustos para justificar sus decisiones ante usuarios expertos o procesos de auditoría [10]. En esencia, optimizan configuraciones, pero no comprenden por qué una solución resulta adecuada.

La integración de Modelos de Lenguaje Grandes (LLMs) introdujo una nueva etapa al permitir interpretar instrucciones en lenguaje natural y generar código de ML de manera iterativa [25]. Paradigmas como ReAct [28] incorporaron capacidades de razonamiento, planificación y uso de herramientas externas, mientras que sistemas como PandasAI demostraron su utilidad para asistir procesos analíticos dinámicos. No obstante, delegar la construcción completa de un pipeline a un único agente LLM presenta problemas de escalabilidad, consistencia y confiabilidad debido a la saturación del contexto, las alucinaciones y la falta de determinismo [15], [13].

Como respuesta a estas limitaciones surgieron los Sistemas Multiagente (MAS), donde la complejidad se distribuye entre agentes especializados con responsabilidades diferenciadas [1]. Frameworks como AutoGen [26] y MetaGPT [14] demostraron que la colaboración entre agentes permite incorporar mecanismos de revisión y validación cruzada. En el ámbito de AutoML, propuestas como AutoML-Agent [24] y MLZero [8] evidenciaron que la cooperación multiagente puede mejorar la exploración del espacio de soluciones frente a enfoques monolíticos.

Paralelamente, el aprendizaje por refuerzo (RL) ha demostrado ser una alternativa prometedora para automatizar la construcción de pipelines mediante la formulación del problema como un Proceso de Decisión de Markov (MDP). Trabajos como DeepLine [12], Auto-CASH [20] y Auto-DP [29] demostraron que los agentes pueden aprender políticas capaces de seleccionar componentes y configuraciones de pipelines superiores a estrategias de búsqueda exhaustiva. Sin embargo, estos métodos permanecen enfocados principalmente en la optimización de métricas y presentan una integración limitada con conocimiento semántico y mecanismos de explicación.

Esta evolución revela una brecha fundamental: los agentes basados en LLM poseen capacidades de razonamiento, pero requieren restricciones formales que garanticen coherencia; los algoritmos de RL pueden descubrir estrategias eficientes, pero funcionan como optimizadores opacos; y las ontologías como OASIS v2 [4] proporcionan estructuras formales de conocimiento, aunque tradicionalmente han permanecido desconectadas de los sistemas AutoML operativos.

La presente propuesta busca cerrar esta brecha mediante la integración de estas tres vertientes. La ontología actúa como una capa de conocimiento que guía y restringe las decisiones del sistema; el RL funciona como un mecanismo de aprendizaje de estrategias óptimas a partir de miles de simulaciones; y la arquitectura multiagente coordina la planificación, ejecución, validación y explicación del proceso. De esta manera, AutoML evoluciona desde un optimizador automático de métricas hacia un sistema autónomo, trazable y explicable, capaz no solo de construir pipelines eficientes, sino también de justificar sus decisiones y comunicar el razonamiento detrás de sus resultados.

3. Arquitectura

La arquitectura propuesta rompe el paradigma convencional del AutoML al integrar una dualidad operativo-cognitiva inspirada en los modelos de procesamiento humano. El sistema no aborda la optimización como una tarea lineal de fuerza bruta, sino como un ciclo continuo de adquisición y

explotación de conocimiento estructurado. Para ello, el diseño se divide en dos fases estructurales. Una fase de simulación agéntica y generación de experiencia estocástica, orientada al descubrimiento empírico y al meta-aprendizaje conceptual. Y una fase de inferencia determinista, aprendizaje jerárquico y explicabilidad, responsable de materializar soluciones óptimas, estables y completamente comprensibles para los tomadores de decisiones estratégicas.

Esta separación garantiza un uso eficiente de las capacidades cognitivas de los LLMs. En lugar de forzar a los agentes a razonar en tiempo real el sistema traslada el esfuerzo intelectual denso a la fase de simulación offline. En consecuencia, el conocimiento formal se consolida de forma determinista y axiomática antes de interactuar con el cliente, permitiendo que la fase de producción opere con una velocidad analítica y una rigurosidad matemática sin precedentes en la literatura de sistemas autónomos.

3.1 Simulación Agéntica y Generación de Experiencia Estocástica

Esta etapa se fundamenta en un marco de conocimiento estricto, mitigando la naturaleza probabilística de los LLMs. La ontología OASIS v2 [4] y complementos semánticos estructurados como DMOP (Data Mining Optimization Ontology) [18] o ML-Schema [21] proveen las reglas axiomáticas del sistema. Estas ontologías definen dependencias estrictas, evitando que el sistema experimente caminos metodológicos inválidos. Por ejemplo, deducen lógicamente que una codificación *One-Hot* es catastrófica para columnas de alta cardinalidad, o imponen que la aplicación de métodos de similitud euclidiana exige un escalado previo de las variables.

El propósito de esta fase es la exploración estocástica profunda sobre diversos datasets (figura 1) para generar trayectorias empíricas y definir los valores de recompensa que guiarán el aprendizaje por refuerzo.

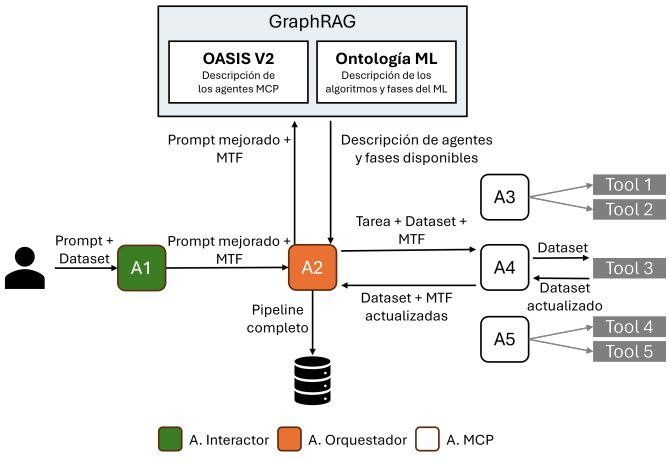


Figura 1. Arquitectura de simulación de pipelines de ML basada en agentes MCP y GraphRAG. Los pipelines son generados mediante orquestación de agentes y exploración aleatoria, con sus ejecuciones almacenadas en una base de conocimiento para aprendizaje posterior. MTF hace referencia a las metacaracterísticas de los datasets.

El flujo de ejecución de esta parte consiste en:

1. **Extracción de Metacaracterísticas y Estado (MDP):** El agente interactor (A1) recibe el conjunto de datos junto con el prompt del usuario que describe los requerimientos y el contexto del problema. A partir de esta entrada, A1 extrae las metacaracterísticas estadísticas (como dimensionalidad, correlaciones, balance de clases, entre otras) para conformar el vector numérico de estado inicial s de un Proceso de Decisión de Markov (MDP) [22], transfiriéndolo al orquestador (A2).
2. **Enrutamiento Ontológico Predictivo vía GraphRAG:** El Agente orquestador (A2) recibe el vector de estado y consulta semánticamente el grafo de conocimiento mediante GraphRAG. El motor de inferencia procesa las metacaracterísticas y activa restricciones axiomáticas en tiempo real, reduciendo el espacio de decisiones al bloquear agentes o transformaciones incompatibles con el dominio, como aplicar SVM a datos de tipo texto. La ontología devuelve así un conjunto restringido y seguro de fases y subtareas, garantizando la viabilidad de las etapas posteriores del pipeline.
3. **Delegación de tareas a agentes MCP:** Una vez delimitadas las opciones válidas, el orquestador (A2) delega subtareas atómicas (como imputaciones de datos, normalización, escalamiento, reducción de dimensionalidad) a agentes obreros especializados (A3 - A5). La interacción operativa y la generación de código se realiza bajo el *Model Context Protocol* (MCP) utilizando herramientas externas (Tool 1-6). Este protocolo actúa como una capa de abstracción que encapsula la ejecución de código, validando su sintaxis, previniendo la corrupción del entorno computacional y devolviendo al flujo central objetos JSON estandarizados.
4. **Exploración Híbrida y Bucle de Retroalimentación:** Para evitar que los agentes se sesguen hacia secuencias convencionales, la toma de decisiones emplea una política híbrida de exploración (ϵ-greedy): con probabilidad p se ejecuta la acción planificada por el LLM, y con $1-p$ se selecciona una acción aleatoria pero válida según las restricciones de la ontología. Cada acción ejecutada transforma el dataset y actualiza su vector de metacaracterísticas (s'), permitiendo al orquestador (A2) evaluar el nuevo estado y seleccionar el siguiente paso hasta completar el pipeline. Al concluir, el desempeño del modelo (*accuracy*, *precision*, *recall*) se asigna como una recompensa retardada (r) que evalúa la calidad global del pipeline.

3.2 Ejecución Determinista, Aprendizaje Jerárquico y Explicabilidad

Con la base de datos poblada por millones de iteraciones de experiencia empírica, el sistema entrena un esquema de aprendizaje por refuerzo profundo jerárquico (Deep HRL). Un agente primario aprende a dictar la estructura óptima del pipeline (las herramientas a utilizar), mientras un agente secundario ajusta los hiperparámetros finos condicionados al

modelo de ML y a las metacaracterísticas del dataset. Este esquema se construiría siguiendo un enfoque de recompensas multiobjetivo utilizando técnicas de vanguardia como las divergencias seccionadas estudiadas en [5], las cuales permiten acelerar el tiempo de cálculo cuando se tiene más de una recompensa asignada.

A diferencia de la simulación iterativa, la producción (figura 2) es un proceso directo, eficaz y altamente explicativo:

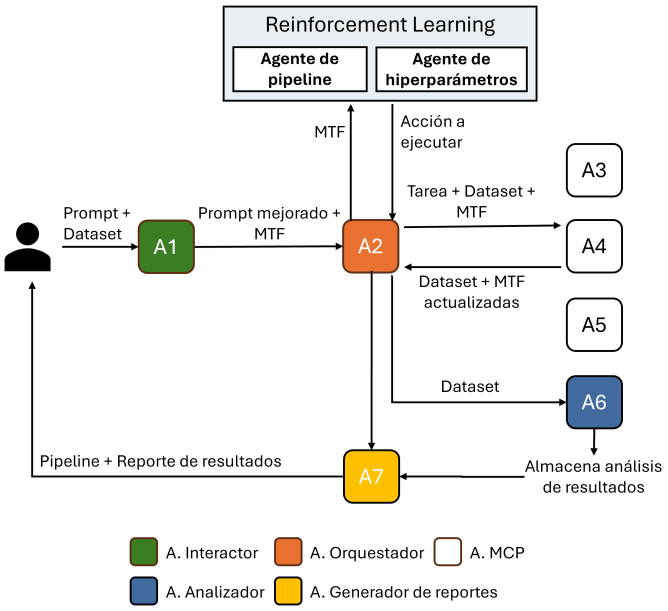


Figura 2. Arquitectura de ejecución del sistema AutoML multiagéntico. El usuario proporciona un dataset y un prompt. A partir de esta información, un agente RL planifica la ejecución y optimiza los hiperparámetros, el orquestador ejecuta el pipeline y genera análisis intermedios, y un agente de reportes sintetiza el proceso, las decisiones tomadas y los principales hallazgos. MTF hace referencia a las metacaracterísticas de los datasets.

1. **Ingesta y Refinamiento del Prompt:** El usuario final interactúa con la plataforma suministrando su conjunto de datos crudo junto con un prompt donde detalla el contexto operativo de los datos, las métricas de negocio prioritarias y el formato de la explicabilidad del pipeline. Un agente interactor (A1) procesa esta entrada, refinando los requerimientos del usuario, reduciendo ambigüedades y clarificando sus objetivos. Posteriormente, transforma estas especificaciones a un formato semántico estandarizado, minimizando errores de interpretación antes de iniciar la ejecución del proceso. Además de calcular las metacaracterísticas del conjunto de datos.
2. **Aplicación de la Política Óptima HRL:** El orquestador (A2) recibe el problema estructurado y, en lugar de iniciar un ciclo de búsqueda costoso, consulta de forma directa la política entrenada del agente HRL. El agente principal determina instantáneamente la configuración del pipeline idóneo basándose en las metacaracterísticas, mientras que el secundario calcula la configuración óptima de hiperparámetros. El plan de ejecución resultante se ejecuta de manera determinista a través de los agentes MCP.
3. **Puntos de Control Estratégicos de XAI:** La arquitectura rompe el secretismo de las cajas negras tradicionales mediante la activación mandatoria de un agente analizador (A6) en *checkpoints* específicos del flujo del pipeline. Este agente no interviene en transformaciones triviales, sino en fases de impacto estructural como cuando el dataset se encuentra limpio y listo para realizar un análisis exploratorio de datos (EDA), cuando se ha aplicado una reducción de dimensionalidad, o cuando se ha entrenado un modelo de ML. El analizador ejecuta algoritmos estadísticos y técnicas de Inteligencia Artificial Explicable (XAI), calculando valores SHAP globales y locales para estimar la importancia de las características. Asimismo, el agente contrasta las variables más influyentes identificadas por SHAP, evaluando si las relaciones explotadas por el modelo reflejan posibles dependencias causales plausibles del dominio o meras correlaciones estadísticas del dataset.
4. **Explicación Narrativa y Auditoría Final:** Una vez consolidado el pipeline y validados los modelos, todos los artefactos, resultados gráficos generados por el agente analizador se transfieren de forma integrada al agente generador de reportes (A7). Este componente cognitivo final procesa los registros técnicos interpretando el comportamiento global del sistema para redactar de forma automatizada un informe en lenguaje natural. El reporte detalla la justificación de cada acción, la relevancia predictiva de las variables y el cumplimiento normativo, entregando al usuario una solución eficaz y auditable.

4. Desafíos principales

La implementación de un sistema multiagéntico de esta complejidad plantea importantes desafíos científicos y de ingeniería. La evolución de AutoML desde enfoques basados en optimización numérica hacia sistemas dinámicos, adaptativos y explicables requiere abordar aspectos clave relacionados con la infraestructura, la coherencia del proceso de toma de decisiones y la gestión del conocimiento. Resolver estos retos será fundamental para garantizar la aplicación del sistema en entornos empresariales donde la confiabilidad y la trazabilidad sean requisitos esenciales.

El primer desafío crítico se relaciona con la elevada demanda computacional e infraestructural requerida durante la fase de simulación. La ejecución de miles de iteraciones de pipelines sobre diversos datasets, junto con el uso de LLMs para ajustar planes y orientar la exploración, puede generar costos operativos elevados. Abordar este desafío requiere el uso de infraestructuras de cómputo de altos recursos, mecanismos de optimización del uso del contexto y entornos aislados en donde las métricas de cómputo como el tiempo de ejecución no se vean alteradas por otros procesos.

Un segundo desafío se centra en la escalabilidad y actualización continua del grafo de conocimiento basado en GraphRAG y la ontología OASIS v2. A medida que el sistema incorpora nuevos agentes, y algoritmos de aprendizaje automático, la

complejidad del grafo aumenta considerablemente. Esto puede afectar la eficiencia de los procesos de inferencia y consulta, por lo que será necesario desarrollar estrategias de organización del conocimiento, indexación eficiente y actualización dinámica que permitan mantener tiempos de respuesta adecuados y evitar simulaciones excesivamente largas.

El tercer desafío técnico está relacionado con la estabilidad y capacidad de generalización del esquema de HRL. La coordinación entre un agente encargado de definir la estructura general del pipeline y otro agente orientado a optimizaciones específicas, como la selección de hiperparámetros, introduce dificultades asociadas al entrenamiento y la estabilidad del aprendizaje. Para lograr que las políticas aprendidas en simulación se adapten correctamente a nuevos conjuntos de datos, será necesario diseñar mecanismos de recompensa multiobjetivo que equilibren aspectos como el desempeño predictivo, y la eficiencia computacional.

5. Conclusiones y trabajos futuros

En conclusión, este artículo de posición presenta un enfoque de AutoML multiagéntico orientado a superar los paradigmas actuales mediante la integración de aprendizaje por refuerzo, ontologías y agentes MCP para la generación automática de pipelines de ML trazables y auditables. Al incorporar mecanismos de explicabilidad y auditoría a lo largo de todo el proceso, la propuesta trasciende la optimización tradicional de métricas de desempeño, avanzando hacia una automatización más transparente, confiable y adaptable. De esta forma, el sistema plantea una visión de AutoML autónomo y explicable, capaz de justificar sus decisiones y comunicar el razonamiento asociado a los resultados obtenidos.

Las implicaciones prácticas de este enfoque pueden ser especialmente relevantes en entornos empresariales, donde la confiabilidad y la adaptabilidad son factores críticos. El sistema podría evolucionar no solo para generar y ejecutar pipelines de forma autónoma, sino también para operar bajo plantillas definidas por el usuario, razonar semánticamente sobre la pertinencia de determinadas técnicas según el contexto de los datos y aplicar restricciones específicas del dominio. Por ejemplo, en escenarios médicos podría identificar situaciones donde la generación de datos sintéticos implique riesgos o introduzca sesgos no deseados. Asimismo, permitiría al usuario definir qué etapas del pipeline deben ejecutarse y establecer puntos de control donde se realicen análisis estadísticos adicionales que enriquezcan el reporte final.

El objetivo futuro es iniciar la construcción del sistema descrito en este artículo y realizar experimentos que permitan evaluar su desempeño y viabilidad en escenarios reales, analizando su capacidad para generar pipelines de ML eficaces, explicables y alineados con las necesidades de los usuarios.

Referencias

[1] P. G. Balaji and D. Srinivasan, "An introduction to multi-agent systems," *Studies in Computational Intelligence*, vol. 310, pp. 1–27, 2010.

[2] M. Baratchi, C. Wang, S. Limmer, J. N. van Rijn, H. Hoos, T. Bäck, and M. Olhofer, "Automated machine learning: Past, present and future," *Artif. Intell. Rev.*, vol. 57, p. 88, 2024.

[3] R. Barbudo, S. Ventura, and J. R. Romero, "Eight years of AutoML: Categorisation, review and trends," *Knowl. Inf. Syst.*, 2023.

[4] G. Bella, D. Cantone, C. F. Longo, M. N. Asmundo, and D. F. Santamaria, "The ontology for agents, systems and integration of services: OASIS version 2," *Applied Ontology*, 2024.

[5] B. Debes and T. Tuytelaars, "Multivariate distributional reinforcement learning using sliced divergences," 2026.

[6] A. Doke and M. Gaikwad, "Survey on automated machine learning (automl) and meta learning," in *Proc. ICCCNT*. Kharagpur, India: IEEE, 2021, p. 5.

[7] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola, "AutoGluon-tabular: Robust and accurate AutoML for structured data," 2020.

[8] H. Fang, B. Han, N. Erickson *et al.*, "MLZero: A multi-agent system for end-to-end machine learning automation," 2025.

[9] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter, "Auto-sklearn: Efficient and robust automated machine learning," in *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019, pp. 113–134.

[10] O. E. Gundersen and S. Kjensmo, "State of the art: Reproducibility in artificial intelligence," in *Proc. AAAI*, 2018, pp. 1644–1651.

[11] X. He, K. Zhao, and X. Chu, "AutoML: A survey of the state-of-the-art," *Knowl.-Based Syst.*, vol. 212, p. 106622, 2021.

[12] Y. Heffetz, R. Vainshtein, G. Katz, and L. Rokach, "DeepLine: AutoML tool for pipelines generation using deep reinforcement learning," in *Proc. ACM SIGKDD*, 2020, pp. 2103–2113.

[13] S. Hong, Y. Lin, B. Liu *et al.*, "Data interpreter: An LLM agent for data science," in *Findings of ACL*, 2024, pp. 19 796–19 821.

[14] S. Hong, M. Zhuge, J. Chen, X. Zheng *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. ICLR*, 2024.

[15] Q. Huang, J. Vora, P. Liang, and J. Leskovec, "MLAgentBench: Evaluating language agents on machine learning experimentation," *Proc. Mach. Learn. Res.*, vol. 235, pp. 20 271–20 309, 2023.

[16] F. Hutter, L. Kotthoff, and J. Vanschoren, Eds., *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

[17] H. Jin, Q. Song, and X. Hu, "Auto-Keras: An efficient neural architecture search system," in *Proc. ACM SIGKDD*, 2019, pp. 1946–1956.

[18] C. M. Keet, A. Ławrynowicz, C. d'Amato, A. Kalousis, P. Nguyen, R. Palma, R. Stevens, and M. Hilario, "The data mining optimization ontology," *Journal of Web Semantics*, vol. 32, pp. 43–53, 2015.

[19] L. Kotthoff, C. Thornton, H. H. Hoos, F. Hutter, and K. Leyton-Brown, "Auto-WEKA 2.0: Automatic model selection and hyperparameter optimization in WEKA," *J. Mach. Learn. Res.*, vol. 18, pp. 1–5, 2017.

[20] T. Mu, H. Wang, C. Wang, Z. Liang, and X. Shao, "Auto-CASH: A meta-learning embedding approach for autonomous classification algorithm selection," *Information Sciences*, vol. 591, pp. 344–364, 2022.

[21] G. C. Publio, D. Esteves, A. Ławrynowicz, P. Panov, L. Soldatova, T. Soru, J. Vanschoren, and H. Zafar, "ML-Schema: Exposing the semantics of machine learning with schemas and ontologies," 2018.

[22] L. Sabbioni, F. Corda, and M. Restelli, "Stepsize learning for policy gradient methods in contextual markov decision processes," in *Proc. ECML PKDD*, 2023, pp. 506–523.

[23] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms," in *Proc. ACM SIGKDD*, 2013, pp. 847–855.

[24] P. Trirat, W. Jeong, and S. J. Hwang, "AutoML-Agent: A multi-agent LLM framework for full-pipeline AutoML," 2025.

[25] H. Wang, J. Gong, H. Zhang, J. Xu, and Z. Wang, "AI agentic programming: A survey of techniques, challenges, and opportunities," *ACM Computing Surveys*, 2025.

[26] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023.

[27] X. Wu, L. Xiao, Y. Sun, J. Zhang, T. Ma, and L. He, "A survey of human-in-the-loop for machine learning," 2022.

[28] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[29] Y. Zhang, N. Wang, Q. Zeng, and L. Li, "Automating data preparation pipeline efficiently via monte carlo tree search," *Information Sciences*, vol. 724, p. 122730, 2026.

[30] L. Zou, "Meta-learning basics and background," in *Meta-Learning*. Elsevier, 2025.